

Build & Configuration Guide

DigiMenu

1. Overview

DigiMenu2 adalah aplikasi digital menu berbasis web untuk restoran/kafe yang dibangun dengan **Laravel 12** di sisi backend dan **Vite + Tailwind CSS v4** di sisi frontend. Aplikasi ini mendukung alur pemesanan dari pelanggan (scan menu, pilih item, checkout), manajemen kasir (status meja), dan panel admin (CRUD menu, CRUD meja, laporan penjualan Excel).

2. Tech Stack

Layer	Teknologi
Backend Framework	Laravel 12 (PHP ^8.2)
Database	SQLite (default), dapat diganti MySQL/PostgreSQL
Frontend Build Tool	Vite 7 + laravel-vite-plugin
CSS Framework	Tailwind CSS v4
HTTP Client (JS)	Axios
Excel Export	Maatwebsite/Laravel-Excel ^3.1
Auth	Laravel built-in Auth (session-based)
Queue / Cache	Database driver (default)

3. System Requirements

- **PHP** ^8.2 (dengan ekstensi: pdo, pdo_sqlite, mbstring, openssl, tokenizer, xml, ctype, json)
- **Composer** ^2.x
- **Node.js** (versi LTS terbaru direkomendasikan) + **npm**
- **Git**

4. Instalasi & Setup

4.1 Clone Repository

```
bash
git clone <url-repo> DigiMenu2
cd DigiMenu2
```

4.2 Cara Cepat (One-Command Setup)

Project ini menyediakan Composer script setup yang mengotomatiskan semua langkah:

```
bash
```

```
composer run setup
```

Script ini secara berurutan akan menjalankan:

1. composer install — install semua PHP dependencies
2. Copy .env.example ke .env (jika belum ada)
3. php artisan key:generate — generate APP_KEY
4. php artisan migrate --force — jalankan semua migrasi database
5. npm install — install Node.js dependencies
6. npm run build — compile aset frontend (CSS/JS)

4.3 Setup Manual (Step by Step)

Jika ingin kontrol lebih, lakukan langkah berikut satu per satu:

```
# 1. Install PHP dependencies
```

```
composer install
```

```
# 2. Salin file konfigurasi environment
```

```
cp .env.example .env
```

```
# 3. Generate application key
```

```
php artisan key:generate
```

```
# 4. Buat file database SQLite (jika belum ada)
```

```
touch database/database.sqlite
```

```
# 5. Jalankan migrasi database
```

```
php artisan migrate
```

```
# 6. (Opsional) Seed data produk awal
```

```
php artisan db:seed
```

```
# 7. Install Node.js dependencies
```

```
npm install
```

```
# 8. Build aset frontend
```

```
npm run build
```

4.4 Membuat Admin User

Aplikasi menggunakan autentikasi berbasis username (bukan email). Buat user admin via Tinker:

```
bash
```

```
php artisan tinker
```

```
php
```

```
App\Models\User::create([
    'name' => 'Admin',
    'username' => 'admin',
    'email' => 'admin@example.com',
    'password' => bcrypt('password_anda'),
]);
```

5. Konfigurasi Environment (.env)

File .env adalah satu-satunya tempat konfigurasi runtime. Salin dari .env.example dan sesuaikan nilai-nilai berikut:

5.1 Konfigurasi Dasar Aplikasi

```
env
```

```
APP_NAME=DigiMenu2
APP_ENV=local      # Ganti ke 'production' saat deploy
APP_KEY=           # Di-generate otomatis oleh artisan key:generate
APP_DEBUG=true     # Set ke 'false' di production
APP_URL=http://localhost
```

5.2 Konfigurasi Database

Default project menggunakan **SQLite** yang tidak memerlukan server database terpisah:

```
env
```

```
DB_CONNECTION=sqlite
# File database ada di: database/database.sqlite
```

Untuk beralih ke **MySQL**, ubah konfigurasi berikut:

```
env
```

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=digimenu2
DB_USERNAME=root
DB_PASSWORD=your_password
```

5.3 Konfigurasi Session, Cache & Queue

Secara default semua driver menggunakan database (tidak memerlukan Redis atau layanan eksternal):

env

```
SESSION_DRIVER=database
SESSION_LIFETIME=120

CACHE_STORE=database
QUEUE_CONNECTION=database
```

5.4 Konfigurasi Mail (Opsional)

Default menggunakan driver log (email ditulis ke log file, tidak benar-benar terkirim):

env

```
MAIL_MAILER=log
```

Untuk menggunakan SMTP sebenarnya:

env

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.provider.com
MAIL_PORT=587
MAIL_USERNAME=your_email
MAIL_PASSWORD=your_password
MAIL_FROM_ADDRESS=noreply@yourdomain.com
MAIL_FROM_NAME="${APP_NAME}"
```

6. Menjalankan Aplikasi (Development)

bash

```
composer run dev
```

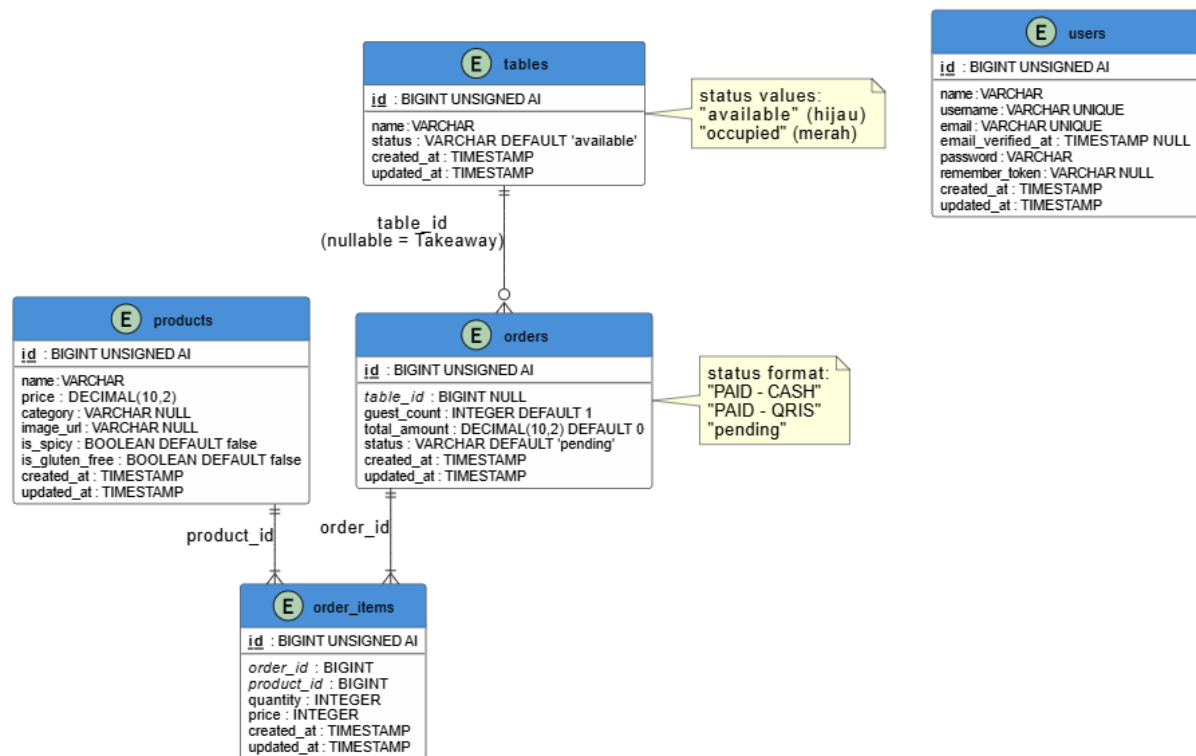
Perintah ini menjalankan empat proses secara bersamaan menggunakan concurrently:

Proses	Perintah	Keterangan
server	php artisan serve	Web server Laravel di port 8000
queue	php artisan queue:listen	Memproses antrian background job
logs	php artisan pail	Live tail log aplikasi
vite	npm run dev	Dev server Vite dengan HMR

Akses aplikasi di: <http://localhost:8000>

7. Struktur Database

Diagram Relasi Tabel



Penjelasan Kolom Penting

- orders.table_id**: Nullable — bernilai null untuk pesanan jenis **Takeaway**.
- orders.status**: Format string "PAID - CASH" atau "PAID - QRIS" sesuai metode pembayaran.
- tables.status**: Nilai available (hijau) atau occupied (merah), digunakan untuk tampilan real-time di halaman pemilihan meja.
- products.is_spicy / is_gluten_free**: Boolean flag untuk filter menu di halaman pelanggan.

Urutan Migrasi

create_users_table	→ users, password_reset_tokens, sessions
create_cache_table	→ cache, cache_locks
create_jobs_table	→ jobs, job_batches, failed_jobs
create_tables_table	→ tables

create_products_table → products

create_orders_table → orders

create_order_items_table → order_items

add_status_to_tables_table → menambah kolom status ke tabel tables

add_username_to_users_table → menambah kolom username ke tabel users

8. Struktur Routing

Public Routes (tanpa autentikasi)

Method	URL	Keterangan
GET	/	Halaman pilih meja / tipe pesanan
GET	/menu	Halaman menu pelanggan
GET	/payment	Halaman pembayaran
POST	/process-checkout	Proses checkout order
GET	/order/{id}/receipt	Halaman struk digital
GET	/kasir/meja	Halaman kasir — lihat status meja
POST	/kasir/meja/toggle/{id}	Toggle status meja (available/occupied)

Auth Routes

Method	URL	Keterangan
GET	/admin/login	Halaman form login admin
POST	/admin/login	Proses login
POST	/admin/logout	Proses logout

Admin Routes (middleware: auth)

Method	URL	Keterangan
GET	/admin/dashboard	Dashboard statistik & grafik
GET	/admin/menu	Daftar menu
POST	/admin/menu/store	Tambah menu baru

PUT	/admin/menu/update/{id}	Edit menu
DELETE	/admin/menu/delete/{id}	Hapus menu
GET	/admin/table	Daftar meja
POST	/admin/table/store	Tambah meja
PUT	/admin/table/update/{id}	Edit meja
DELETE	/admin/table/delete/{id}	Hapus meja
GET	/admin/export-excel	Download laporan penjualan (.xlsx)

9. Alur Aplikasi

Alur Pelanggan

1. Buka / → pilih meja (dine-in) atau takeaway
2. Diarahkan ke /menu → browse dan tambahkan item ke keranjang
3. Checkout ke /payment → pilih metode pembayaran (Cash/QRIS)
4. POST ke /process-checkout → order disimpan ke database, meja diubah ke status occupied
5. Redirect ke /order/{id}/receipt → tampil struk digital

Alur Admin

1. Login ke /admin/login menggunakan username + password
2. Dashboard /admin/dashboard menampilkan total pesanan, pendapatan, jumlah menu, jumlah meja, dan grafik pesanan per bulan
3. Kelola menu produk dan meja melalui halaman CRUD masing-masing
4. Export laporan penjualan ke file .xlsx melalui /admin/export-excel

Alur Kasir

1. Buka /kasir/meja untuk melihat semua meja beserta statusnya
2. Toggle status meja (available ↔ occupied) via POST ke /kasir/meja/toggle/{id}

10. Frontend Build

Konfigurasi Vite ada di vite.config.js:

- **Entry point:** resources/css/app.css dan resources/js/app.js

- **Plugin:** laravel-vite-plugin (untuk asset fingerprinting & hot reload) + @tailwindcss/vite (untuk Tailwind v4)
- **Watch ignored:** storage/framework/views/** (folder cache Blade, tidak perlu di-watch)

bash

```
# Development (HMR aktif)
npm run dev

# Production build (minified, fingerprinted)
npm run build
```

11. Fitur Export Excel

Laporan penjualan di-generate menggunakan class App\Exports\PenjualanExport dengan library Maatwebsite/Laravel-Excel. Format file: .xlsx.

Kolom yang diekspor:

1. Pesanan Ke (format: #ORD-0001)
2. Tanggal (format: Y-m-d H:i)
3. ID Meja (Meja X atau Takeaway)
4. Jumlah Tamu
5. Total Harga (IDR)
6. Status Pesanan

Nama file otomatis mengikuti format: Laporan_Penjualan_DigiMenu_YYYY-MM-DD.xlsx

12. Menjalankan Test

bash

```
composer run test
```

Atau langsung:

bash

```
php artisan config:clear
php artisan test
```

File test tersimpan di tests/Feature/ dan tests/Unit/.

13. Catatan Deployment (Production)

1. Set APP_ENV=production dan APP_DEBUG=false di .env
2. Jalankan npm run build untuk menghasilkan aset yang sudah dioptimasi

3. Jalankan `php artisan config:cache && php artisan route:cache && php artisan view:cache` untuk performa optimal
4. Pastikan direktori `storage/` dan `bootstrap/cache/` dapat ditulis oleh web server
5. Buat symlink storage public: `php artisan storage:link`
6. Untuk produksi, pertimbangkan mengganti database dari SQLite ke MySQL/PostgreSQL
7. Gunakan `php artisan queue:work` (atau Supervisor) sebagai daemon untuk memproses antrian